SLA-AWARE CLOUD STORAGE RESOURCE ALLOCATION OPTIMIZER

Simple Technical Guide for System Foundations and Allocation Engine

1. System Foundations & Overview

The **SLA-Aware Cloud Storage Resource Allocation Optimizer** is an automated decision tool designed to help cloud tenants select the best cloud storage tier for their applications. Cloud providers offer different storage products—such as fast SSD Block Storage, versatile File Storage, and low-cost Object Storage. Manual selection is difficult because fast, reliable storage is expensive, while low-cost storage is slower and less reliable. Selecting storage manually often leads to over-spending or unexpected performance outages.

The optimizer solves this problem by taking the user's workload requirements and processing them through a two-stage decision engine: first filtering out non-compliant storage options, and then scoring the remaining options using a weighted multi-objective heuristic algorithm.

2. The 4 Core System Parameters

The system evaluates storage options using four simple parameters:

1. **Max Tolerable Access Latency (L_req in ms)**: The maximum response time an application can tolerate. Hardware latency dictates speed (SSD Block = 2.0 ms, NAS File = 10.0 ms, Object = 50.0 ms). Any storage tier slower than L_req is physically disqualified.

2. **Availability SLA Target (A_req in %)**: The minimum required uptime guarantee. SLA values determine allowable downtime per year (99.0% allows 87.6 hours; 99.999% allows only 5.26 minutes). Tiers offering lower uptime than A_req are disqualified.

3. **Cost Optimization Weight (α, Alpha)**: A slider between 0.0 and 1.0 that sets priority between cost savings and uptime reliability. Setting Alpha = 1.0 focuses entirely on cost

reduction; setting Alpha = 0.0 focuses entirely on high availability; setting Alpha = 0.5 balances both equally (companion weight $\beta = 1.0 - \alpha$).

4. **Monthly Budget Constraint (B in $, Optional)**: An optional monthly spending limit. Any tier whose total monthly cost exceeds B is disqualified.

3. System Conceptual Architecture & Flowchart

The system processes requests in three logical phases: capturing user inputs, filtering non-compliant options, and scoring the remaining options. Figure 1 illustrates this conceptual framework.

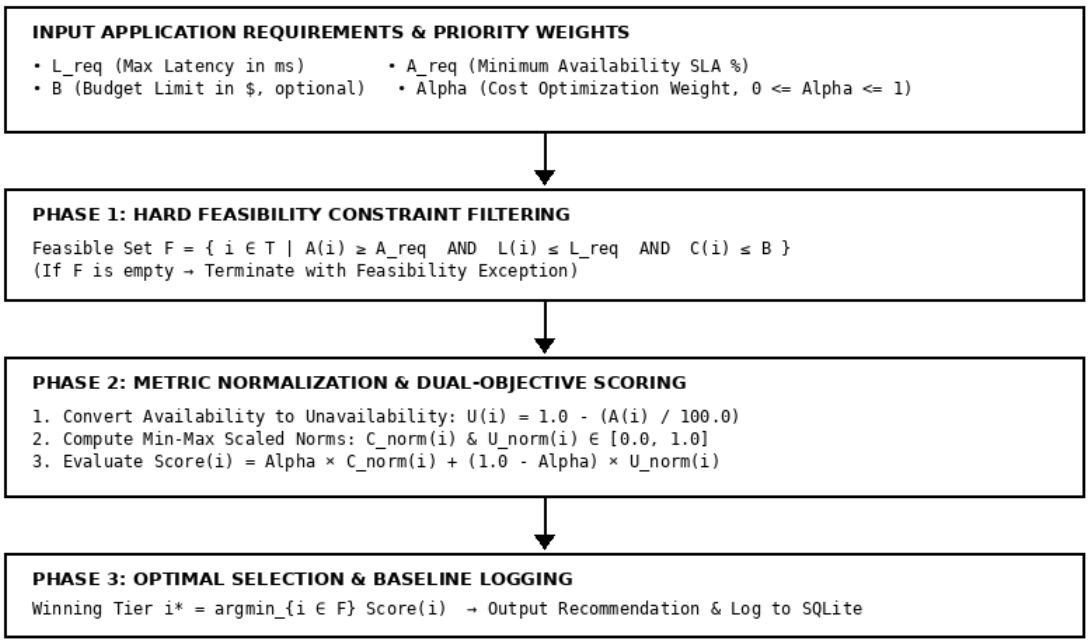

Figure 1: Conceptual Framework showing parameter relationships and decision pipeline

Figure 2 shows the operational flowchart detailing how requests pass from the user interface into the optimization engine, query the SQLite database, apply filtering rules, calculate normalized scores, and log the recommendation transaction into the database.

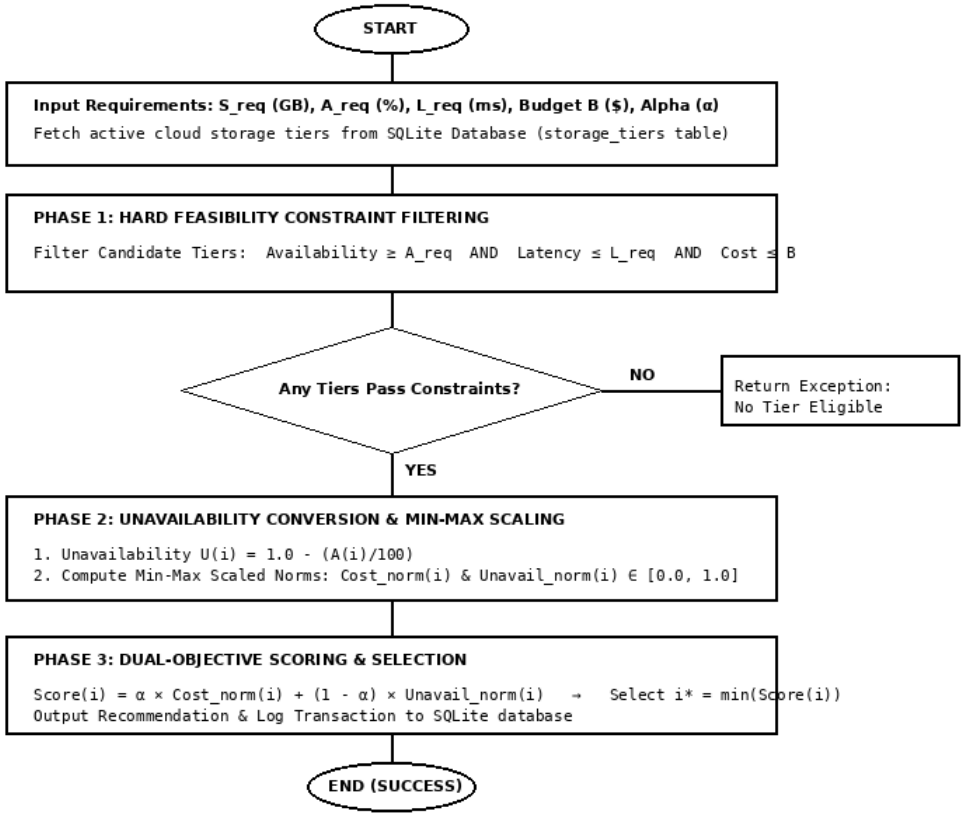

Figure 2: System Algorithm Flowchart showing Constraint Pipeline and Dual Scoring

4. Core Heuristic Algorithm Pseudocode

The recommendation engine executes the **SLA-Aware Dual-Objective Storage Allocation Heuristic Algorithm**. Figure 3 provides a screenshot snapshot of the formal pseudocode logic.

```
Input: Storage Tiers T, Size S_req, Availability Target A_req, Latency Limit L_req, Budget B, Weight α
Output: Optimal Storage Tier i* and Estimated Monthly Cost

FeasibleSet F ← ∅
for each storage tier i in T do
    TotalCost[i] ← UnitCost[i] × S_req
    if Availability[i] ≥ A_req and Latency[i] ≤ L_req then
        if Budget B is set and TotalCost[i] > B then continue
        F ← F ∪ {i}
    end if
end for
if F = ∅ then return Error("No tier meets SLA, Latency, or Budget bounds")
for each tier i in F do
    Unavailability[i] ← 1.0 - (Availability[i] / 100.0)
end for
Compute Cost_norm(i) and Unavail_norm(i) using Min-Max scaling over F
for each tier i in F do
    Score[i] ← α × Cost_norm(i) + (1.0 - α) × Unavail_norm(i)
end for
i* ← argmin_{i ∈ F} Score[i]
return Recommended Tier i*, TotalCost[i*], Availability[i*], Latency[i*]
```

Figure 3: Algorithm 3.1 Pseudocode logic of the core SLA-aware dual-objective heuristic algorithm

When an allocation request runs, the algorithm computes total monthly cost for each tier, filters out non-compliant tiers, converts uptime percentage into unavailability, scales cost and unavailability metrics into a 0.0 to 1.0 range, and evaluates the final weighted score. The tier with the lowest score is selected as the winning recommendation.

5. Simple Step-by-Step Optimization Process

The algorithm follows seven simple, proper steps to generate a recommendation:

* **Step 1 (Sizing)**: Determine the required storage capacity in Gigabytes (S_req).

* **Step 2 (Cost Calculation)**: Calculate total monthly cost for each storage tier: Cost = UnitCost × S_req.

* **Step 3 (Hard Filtering)**: Remove any tier that fails access latency (Latency > L_req), uptime (SLA < A_req), or budget (Cost > B).

* **Step 4 (Unavailability Conversion)**: Convert uptime percentage into unavailability: Unavailability = 1.0 - (SLA / 100).

* **Step 5 (Range Scaling)**: Normalize cost and unavailability into a standard 0.0 to 1.0 scale over eligible tiers.

* **Step 6 (Dual Scoring)**: Calculate final score: Score = Alpha × Cost_norm + (1.0 - Alpha) × Unavail_norm.

* **Step 7 (Selection)**: Select the tier with the lowest score as the winning recommendation and save the record to SQLite.

6. Simple Worked Example (500 GB Request)

Here is a simple numerical walk-through demonstrating how the algorithm selects a tier:

Input Requirements:

* **Storage Size**: 500 GB

* **Availability Target**: 99.0% minimum uptime

* **Latency Limit**: 20.0 ms maximum delay

* **Cost Priority (Alpha)**: 0.7 (70% weight on cost savings, 30% weight on uptime reliability)

Candidate Tiers in Database:

1. **Block Storage (SSD)**: 0.15/GB (75.00/month), 99.999% SLA, 2.0 ms latency.

2. **File Storage (NAS)**: 0.08/GB (40.00/month), 99.99% SLA, 10.0 ms latency.

3. **Object Storage (S3)**: 0.02/GB (10.00/month), 99.0% SLA, 50.0 ms latency.

Step-by-Step Execution:

1. **Constraint Check**: Object Storage latency is 50.0 ms, which exceeds the 20.0 ms limit → **Object Storage is disqualified**. Block Storage (2.0 ms) and File Storage (10.0 ms) pass.

2. **Metric Normalization**: Over eligible tiers (Block & File), Block Storage has higher cost (Cost_norm = 1.0) and lower unavailability (Unavail_norm = 0.0). File Storage has lower cost (Cost_norm = 0.0) and higher unavailability (Unavail_norm = 1.0).

3. **Score Evaluation (Alpha = 0.7, Beta = 0.3)**:

* Block Storage Score = (0.7 × 1.0) + (0.3 × 0.0) = **0.70**

* File Storage Score = (0.7 × 0.0) + (0.3 × 1.0) = **0.30**

4. **Final Decision**: File Storage has the lower score (0.30 vs 0.70) → **File Storage is selected as the winning recommendation!**